SOFT L-TOPOLOGICAL LANGUAGE MODELLING

A Thesis

Submitted in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

by

RITHESH S S (CB.AI.U4AID23031)

ASHWIN J R (CB.AI.U4AID23029)

VISHWA VINAYAK C K (CB.AI.U4AID23051)

PRANAV SIVAKUMAR (CB.AI.U4AID23028)

Team AA03



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled "**Soft L-Topological Language Modelling**" submitted by **Rithesh S S** (CB.AI.U4AID23031), **Ashwin J R** (CB.AI.U4AID23029), **Vishwa Vinayak C K** (CB.AI.U4AID23051), and **Pranav Sivakumar** (CB.AI.U4AID23028) for the award of the **Degree of Bachelor of Technology in Artificial Intelligence and Data Science** under the **Faculty of Engineering** is a bonafide record of the original research work carried out under my guidance and supervision at the "Department of Artificial Intelligence", Amrita Vishwa Vidyapeetham, Coimbatore Campus.

Submitted for the examination held on October 23, 2026.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Dr. Sandhya S. Pai
Thesis Advisor / Assistant Professor,
Department of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS

DECLARATION

We, **Rithesh S S** (CB.AI.U4AID23031), **Ashwin J R** (CB.AI.U4AID23029), **Vishwa Vinayak C K** (CB.AI.U4AID23051), and **Pranav Sivakumar** (CB.AI.U4AID23028), hereby declare that this thesis entitled "**Soft L-Topological Language Modelling**" is the record of original work conducted by us under the guidance of **Dr. Sandhya S. Pai**, Assistant Professor, Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus. To the best of our knowledge, this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or similar title in this or any other University.

Place: Coimbatore **Signature of the Students**
Date: October 23, 2026

COUNTERSIGNED

Dr. K. P. Soman
Professor and Dean,
Department of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

Contents

Acknowledgements ..... **ii**
List of Figures ..... **iii**
List of Tables ..... **iv**
Abbreviations and Symbols ..... **v**
Abstract ..... **vi**
1 Introduction **1**
1.1 Background and Motivation 1
1.2 Core Idea: Multi-Attribute Continuous Logic 1
1.3 Problem Statement and Objectives 2
1.4 Summary of Key Findings and Outline 2
2 Theoretical Foundations and Related Work **3**
2.1 The Standard Transformer and Softmax Bottleneck 3
2.2 Prior Approaches to Sparsity 3
2.3 Soft Set Theory and Continuous Lattices 4
2.4 Triangular Norms and the Nilpotent Advantage 4
3 Proposed Soft L-Topological Architecture **5**
3.1 System Architecture Overview 5
3.2 Bounded Continuous Lattice Embeddings 5
3.3 The Continuous Lattice Sequence Router 6
3.4 Continuous Lattice KV-Caching 7
4 Experimental Setup and Foundational Ablations **8**
4.1 Experimental Methodology and Controlled Parity 8
4.2 Foundational Empirical Ablations (Experiments 1 to 5) 9
5 Large-Scale Benchmarks and Critical Analysis **10**
5.1 Large-Scale Benchmarks (Experiments 6 to 10) 10
5.2 Critical Discussion and Explainability 12
6 Conclusion and Future Scope **13**
6.1 Summary of Contributions 13
6.2 Practical Implications and Future Work 13
A Mathematical Proofs and Specifications **14**
A.1 Mathematical Proofs of Nilpotent Triangular Norm Properties 14
A.2 Automated Unit Testing Suite (19 Passing Tests) 15
A.3 Reproducibility Protocol and Environment Setup 15
References **16**

Acknowledgements

We express our profound gratitude to our project supervisor and mentor, **Dr. Sandhya S. Pai**, Assistant Professor in the Department of Artificial Intelligence, whose pioneering mathematical research on soft lattice topological spaces (Pai & Baiju, 2021) provided the theoretical foundation for this thesis. Her continuous intellectual guidance, mathematical insights, and constructive reviews were invaluable throughout the conception, implementation, and benchmarking of this architecture.

We extend our sincere thanks to **Dr. K. P. Soman**, Professor and Dean, Department of Artificial Intelligence. We also thank our peers, faculty members, and families for their constant encouragement.

RITHESH S S
ASHWIN J R
VISHWA VINAYAK C K
PRANAV SIVAKUMAR
(Team AA03)

List of Figures

4.1 Exp 2: Operator Pareto Frontier. 9
4.2 Exp 4: Threshold τ vs. Sparsity. 9

5.1 Exp 7: Distractor Scaling Curve. 11
5.2 Exp 8: Deductive Transitivity. 11
5.3 Exp 9: High-Volume Scaling. 11
5.4 Exp 10: Encoder MLM Benchmarks. 11

List of Tables

2.1 Canonical Continuous Triangular Norm Families 4

4.1 Architectural Specifications and Parameter Parity Comparison 8

5.1 Experiment 7: Distractor Scaling Needle-in-a-Haystack Results 10
5.2 Experiment 9: High-Volume Autoregressive Scaling Metrics 11

A.1 Comprehensive Unit Testing Suite (19 Passing Automated Tests) 15

Abbreviations and Symbols

Abbreviations

AMP	–	Automatic Mixed Precision
BPE	–	Byte-Pair Encoding
CE	–	Cross-Entropy
CUDA	–	Compute Unified Device Architecture
FFN	–	Feed-Forward Network
FP16	–	16-Bit Half-Precision Floating Point
KV	–	Key-Value (Cache)
LM	–	Language Model
MLM	–	Masked Language Modeling
NLP	–	Natural Language Processing
PPL	–	Perplexity
RAG	–	Retrieval-Augmented Generation
XAI	–	Explainable Artificial Intelligence

Mathematical Symbols

$[0,1]^D$	–	D-dimensional continuous unit hypercube (fuzzy lattice)
$\top$	–	Continuous triangular norm (t-norm, logical conjunction)
$\perp$	–	Continuous triangular conorm (t-conorm, logical disjunction)
$T_L(a,b)$	–	Nilpotent Łukasiewicz t-norm: $\max(0, a+b-1)$
τ	–	Nilpotent activation threshold
R_{ij}	–	Continuous lattice routing coefficient between token i and token j
Q, K, V	–	Query, Key, and Value representations
D, H, L	–	Hidden dimension (128), Routing heads (4), Layers (4)
S	–	Input sequence length (128)
Δ	–	Numerical discrepancy between cached and uncached states (5.06×10^{-7})
ϵ	–	Numerical stabilizer constant (10^{-7})

Abstract

Most modern autoregressive language models rely on scaled dot-product attention followed by softmax normalization. While effective, softmax has an inherent mathematical drawback: because the exponential function never reaches zero, it assigns a non-zero probability to every context token. As a result, irrelevant tokens and context distractors always retain some attention weight, which causes noise leakage and prevents true mathematical sparsity.

In this thesis, we explore the **Soft L-Topological Language Model**, an alternative sequence routing architecture based on continuous soft set theory and fuzzy lattices. Instead of unconstrained vector embeddings and dense softmax weights, tokens are mapped to continuous soft sets bounded in the unit hypercube $[0, 1]^D$. Routing weights between tokens are computed using a thresholded nilpotent Łukasiewicz t-norm ($T_L(a, b) = \max(0, a + b - 1)$). When semantic attribute overlap between two tokens drops below a threshold τ, their connection drops to exactly zero (0.0000). We evaluated this architecture across 10 experiments spanning 6.14 million tokens on WikiText-2 and TinyStories:

- **Perplexity Trade-off:** Soft L-Topology trains stably (validation loss 3.7455 / PPL 42.33 on TinyStories), but does not match the baseline Transformer (loss 3.4866 / PPL 32.67), because softmax produces smoother gradients that better suit cross-entropy loss.
- **Deterministic Sparsity:** In return, Soft L-Topology maintains **55.06% exact-zero routing sparsity** ($R_{ij} \equiv 0.0000$) throughout autoregressive generation.
- **Distractor Immunity:** In 16-distractor haystack stress tests, Soft L-Topology zeroes out 66.9% of distractor connections, cutting irrelevant attention leakage by **49.4% in absolute terms**.
- **Logical Deduction & Masked Modeling:** Soft L-Topology doubles accuracy on multi-step Modus Ponens reasoning (40.0% vs. 20.0%) and achieves lower Masked Language Modeling loss (6.2041 vs. 6.2422) under heavy distractor noise.

Chapter 1

Introduction

1.1 Background and Motivation

Most modern autoregressive language models rely on the Transformer architecture [1], which computes sequence representations via multi-head self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad \text{where softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \tag{1.1}$$

In practice, self-attention has proven exceptionally versatile. However, its core normalization operator, softmax, introduces an inescapable mathematical property: because $\exp(z)$ is strictly positive for any finite logit $z \in \mathbb{R}$, softmax assigns a non-zero probability to every token in the sequence. This means the model always maintains a fully connected attention graph. Every single word in the context window—whether it is an essential syntactic dependency or completely unrelated background text—receives non-zero weight. This design leads to two notable drawbacks:

1. **Distractor Noise Leakage:** Real-world inputs frequently contain conversational filler, unrelated paragraphs, or noisy retrieved context. Because softmax cannot assign an exact zero weight (0.0000), these distractor tokens continually draw attention mass away from relevant facts. Over several stacked layers, this small leakage accumulates and can contribute to factual hallucinations [2].
2. **Lack of True Sparsity:** Computing pairwise attention across a sequence of length S requires $O(S^2)$ memory and operations. Even when the majority of tokens in a prompt have no semantic bearing on the current word, standard attention cannot prune these connections to true zero.

1.2 Core Idea: Multi-Attribute Continuous Logic

In this thesis, we explore an alternative perspective on token interaction. Rather than measuring the angular similarity of unconstrained vectors in $\mathbb{R}^D$ using dot products, we model tokens as collections of continuous semantic attributes bounded inside the unit hypercube $[0, 1]^D$.

Under this view, interaction is defined by attribute satisfaction and logical containment.

When an entity lacks the attributes demanded by a relation, the logical conjunction should drop cleanly to zero. To implement this behavior within a differentiable neural network, we use the **nilpotent Łukasiewicz triangular norm (t-norm)** [3], defined as $T_L(a, b) = \max(0, a + b - 1)$. Whenever the sum of two attribute values $(a + b)$ is less than or equal to 1, the output evaluates to exactly zero (0.0000). By incorporating a threshold τ, this operator functions like a continuous gate: when semantic overlap is insufficient, the connection between tokens is cleanly cut.

1.3 Problem Statement and Objectives

This work examines a concrete question: *Can continuous soft set representations in $[0, 1]^D$ combined with nilpotent triangular norms build a practical sequence router that achieves exact-zero sparsity and distractor resilience without sacrificing fluency?*

To address this, we set four objectives: (1) Formulate a continuous lattice router using bounded convex embeddings, nilpotent Łukasiewicz t-norms, and Jaccard containment; (2) Train a 13.67M-parameter model under strict parameter and compute parity with a standard Transformer on WikiText-2 and TinyStories; (3) Measure distractor resilience under synthetic Needle-in-a-Haystack tests and evaluate logical reasoning on Modus Ponens deduction chains; and (4) Systematically examine trade-offs between exact-zero sparsity, generation speed, and next-token perplexity.

1.4 Summary of Key Findings and Outline

Our experiments highlight four main outcomes: (1) *Perplexity Trade-Off:* Soft L-Topology trails the standard Transformer in perplexity (3.7455 loss / 42.33 PPL vs. 3.4866 loss / 32.67 PPL on TinyStories), as softmax provides smoother gradients for cross-entropy log-loss; (2) *Deterministic Sparsity:* The model maintains **55.06% exact-zero routing** ($R_{ij} \equiv 0.0000$) during autoregressive generation; (3) *Distractor Immunity:* Under 16 injected distractors, it prunes 66.9% of distractor weights to exact zero, reducing attention leakage by **49.4% in absolute terms**; and (4) *Deductive Entailment & Encoder Edge:* It doubles deduction accuracy on Modus Ponens (40.0% vs. 20.0%) and achieves lower Masked Language Modeling loss (6.2041 vs. 6.2422) under distractor noise.

The rest of this thesis is structured as follows: Chapter 2 discusses theoretical foundations; Chapter 3 describes the architecture and lattice KV-caching; Chapter 4 presents setup and baseline ablations on WikiText-2; Chapter 5 details evaluations on TinyStories and critical discussion; Chapter 6 summarizes conclusions and future directions; and Appendix A contains mathematical proofs and test specifications.

Chapter 2

Theoretical Foundations and Related Work

2.1 The Standard Transformer and Softmax Bottleneck

The Transformer architecture [1] maps an input token sequence $(w_1, \ldots, w_S)$ to contextualized representations via multi-head self-attention. For sequence representations $X \in \mathbb{R}^{S \times D}$, linear projection matrices produce Queries (Q), Keys (K), and Values (V): $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. Pairwise compatibility is computed via scaled dot products normalized row-wise by softmax:

$$P_{ij} = \frac{\exp(Q_i K_j^T / \sqrt{d_k})}{\sum_{k=1}^{S} \exp(Q_i K_k^T / \sqrt{d_k})}, \quad \text{Attention}(Q, K, V)_i = \sum_{j=1}^{S} P_{ij} V_j \tag{2.1}$$

Proposition 2.1 (Strict Positivity of Softmax)**.** Let $\mathbf{z} \in \mathbb{R}^S$ be any finite vector of logits. Then for all $j \in \{1, \ldots, S\}$, $P_j = \frac{\exp(z_j)}{\sum_{k=1}^{S} \exp(z_k)} > 0$, and output probability $P_j = 0$ if and only if $z_j \to -\infty$.

In floating-point arithmetic, logits are always finite, so $P_{ij} > 0$ for all token pairs. In long contexts, completely irrelevant words receive small non-zero weights (10^{-3} to 10^{-6}). Across layers, these fractional weights compound, pulling distractor features into hidden states and contributing to factual hallucinations [2].

2.2 Prior Approaches to Sparsity

Researchers have previously explored two main strategies to introduce attention sparsity:

- **Heuristic Structural Sparsity:** Models like Longformer [4] and BigBird [5] restrict attention to fixed sliding windows and dilated patterns. While reducing complexity, these routing graphs are static and cannot adapt to semantic content.
- **Mathematical Activation Sparsity:** Sparsemax [6] projects activations onto the probability simplex to produce hard zeros, but zero-gradient regions outside the active set can

destabilize training. Entmax [7] provides a smoother interpolation via Tsallis entropy, but relies on iterative solvers that add runtime overhead.

2.3 Soft Set Theory and Continuous Lattices

To build a routing mechanism that can disconnect unrelated tokens dynamically, we draw on **Soft Set Theory**, introduced by Molodtsov [8] as a mathematical framework for parameterized uncertainty.

Definition 2.1 (Soft Set)**.** Let U be a universe of objects and E a set of parameters. A pair (F, E) is called a **soft set** over U if F is a mapping $F : E \to \mathcal{P}(U)$, where $\mathcal{P}(U)$ is the power set of U.

Maji et al. [9] extended soft sets to *Fuzzy Soft Sets*, allowing attribute memberships in $[0, 1]$. Later, Pai and Baiju [10] introduced *Soft Lattice Topological Spaces*, connecting soft sets directly to continuous lattice structures. In this work, we bring these principles into sequence modeling: each discrete vocabulary token w is represented as a soft set over the unit hypercube $\mathbf{x} \in [0, 1]^D$, where each coordinate $x_d \in [0, 1]$ indicates the degree to which that token expresses a latent attribute.

2.4 Triangular Norms and the Nilpotent Advantage

To compute logical conjunctions (intersections) on $[0, 1]^D$, we employ **Triangular Norms (t-norms)** [3], which satisfy commutativity, associativity, monotonicity, and boundary conditions ($T(a, 1) = a, T(a, 0) = 0$).

Table 2.1: Canonical Continuous Triangular Norm Families

Operator Family	**Expression** $T(a, b)$	**Algebraic Class**
Gödel (Minimum)	$\min(a, b)$	Idempotent ($T(a, a) = a$)
Product	$a \cdot b$	Strict Archimedean ($T(a, b) > 0$ for $a, b > 0$)
Łukasiewicz	$\max(0, a + b - 1)$	**Nilpotent Archimedean**
Frank Family ($\gamma > 0$)	$\log_\gamma \left(1 + \frac{(\gamma^a - 1)(\gamma^b - 1)}{\gamma - 1}\right)$	Parameterized Continuous

A t-norm is called **nilpotent** if non-zero elements $a, b > 0$ can yield $T(a, b) = 0$. For Łukasiewicz, $T_L(a, b) = \max(0, a + b - 1) = 0 \iff a + b \leq 1$. Unlike Gödel or Product, which stay strictly positive for positive inputs, Łukasiewicz provides a genuine zero floor. Moreover, for $a + b > 1$, its derivatives are constant ($\frac{\partial T_L}{\partial a} = \frac{\partial T_L}{\partial b} = 1.0$), ensuring steady gradient flow in active regions while cutting inactive links to exact zero.

Chapter 3

Proposed Soft L-Topological Architecture

3.1 System Architecture Overview

The Soft L-Topological Neural Network is designed for both autoregressive generation and bidirectional representation learning. Instead of using standard dot-product attention in Euclidean space with softmax, it relies on **bounded continuous soft set representations** and **nilpotent lattice sequence routing**.

The overall forward data flow proceeds as follows:

1. Input discrete tokens $w_t \in \mathcal{V}$ are embedded directly into the continuous unit hypercube $[0,1]^D$.
2. Bounded learnable positional encodings in $[0,1]^D$ are combined with token representations via convex blending.
3. A stack of L identical **Lattice Blocks** processes the representations, each combining a multi-head continuous lattice router, residual connections, and position-wise feed-forward networks (FFN).
4. A final linear projection computes next-token logits over the vocabulary.

3.2 Bounded Continuous Lattice Embeddings

For continuous logic operations to remain mathematically well-defined, token coordinates must stay within the unit hypercube $[0,1]^D$ rather than drifting to arbitrary magnitudes.

3.2.1 Convex Token and Positional Blending

Raw embedding parameters $\widetilde{E} \in \mathbb{R}^{|\mathcal{V}| \times D}$ and positional parameters $\widetilde{P} \in \mathbb{R}^{S \times D}$ are normalized via element-wise sigmoid: $\mathbf{e}_{\text{tok}}(w) = \sigma(\widetilde{E}_w) \in (0,1)^D$ and $\mathbf{e}_{\text{pos}}(t) = \sigma(\widetilde{P}_t) \in (0,1)^D$. Instead of

unconstrained addition, they are combined via **convex blending** with coefficient $\lambda = 0.5$:

$$\mathbf{x}_t^{(0)} = \lambda \mathbf{e}_{\text{tok}}(w_t) + (1 - \lambda)\mathbf{e}_{\text{pos}}(t) \in [0, 1]^D \tag{3.1}$$

Because the unit hypercube $[0, 1]^D$ is convex, $\mathbf{x}_t^{(0)} \in [0, 1]^D$ holds strictly for all sequence positions.

3.2.2 Beta Distribution Initialization

If raw embeddings are initialized with a standard zero-centered Gaussian distribution, applying a sigmoid pushes all coordinate values close to 0.5. To promote diversity among semantic attributes from the start, we initialize embedding weights using an asymmetric **Beta distribution**:

$$\widetilde{E}_{vd} \sim \text{logit}(\text{Beta}(\alpha, \beta)), \quad \alpha = 1.0,\ \beta = 3.0 \tag{3.2}$$

This concentrates initial memberships between 0.1 and 0.4, setting most attribute pairs near or below the nilpotent threshold at initialization.

3.3 The Continuous Lattice Sequence Router

For hidden states $X \in \mathbb{R}^{S \times D}$, the router computes H parallel heads ($d_k = D/H$): Queries $Q^{(h)} = \sigma(XW_Q^{(h)}) \in [0, 1]^{S \times d_k}$, Keys $K^{(h)} = \sigma(XW_K^{(h)}) \in [0, 1]^{S \times d_k}$, and Values $V^{(h)} = XW_V^{(h)} \in \mathbb{R}^{S \times d_v}$. Queries and Keys act as bounded attribute memberships in $[0, 1]^{d_k}$. Values, on the other hand, are kept in unconstrained $\mathbb{R}^{d_v}$ so the model can preserve sufficient expressive capacity.

3.3.1 Thresholded Łukasiewicz Conjunction

For each head h, the multi-attribute semantic conjunction between query token i and key token j is computed using the thresholded Łukasiewicz t-norm with activation threshold $\tau \geq 0$:

$$\mathcal{C}_{ij}^{(h)} = \sum_{d=1}^{d_k} \max(0, Q_{id}^{(h)} + K_{jd}^{(h)} - 1 - \tau) \tag{3.3}$$

Whenever $Q_{id}^{(h)} + K_{jd}^{(h)} \leq 1 + \tau$ across all dimensions, the conjunction $\mathcal{C}_{ij}^{(h)}$ evaluates to exactly **0.0000**.

3.3.2 Continuous Jaccard Relative Containment

To normalize these connection scores while avoiding division by zero, we compute a **continuous Jaccard containment coefficient** by dividing conjunction by the algebraic union (t-conorm):

$$R_{ij}^{(h)} = \frac{\sum_{d=1}^{d_k} \max(0, Q_{id}^{(h)} + K_{jd}^{(h)} - 1 - \tau)}{\sum_{d=1}^{d_k} \left(Q_{id}^{(h)} + K_{jd}^{(h)} - Q_{id}^{(h)} K_{jd}^{(h)} \right) + \epsilon} \tag{3.4}$$

where $\epsilon = 10^{-7}$. This formulation ensures that $0.0000 \leq R_{ij}^{(h)} \leq 1.0000$ always, and $R_{ij}^{(h)} \equiv 0.0000$ whenever the numerator is zero.

3.3.3 Causal Masking and Value Aggregation

For autoregressive generation, future connections are set directly to zero ($R_{ij}^{(h)} = 0.0000$ for $j > i$). Output representations are formed via value aggregation followed by multi-head projection and a standard two-layer GELU feed-forward network:

$$\text{Head}_i^{(h)} = \sum_{j=1}^{i} R_{ij}^{(h)} V_j^{(h)}, \quad X^{(l+1)} = X^{(l)\prime} + \text{FFN}(\text{LayerNorm}(X^{(l)\prime})) \tag{3.5}$$

3.4 Continuous Lattice KV-Caching

During autoregressive decoding, recomputing the entire sequence at each step incurs $O(S^2)$ cost. In the Soft L-Topological model, Key-Value caching stores past continuous keys $K_{\text{past}} \in [0,1]^{t \times d_k}$ and values $V_{\text{past}} \in \mathbb{R}^{t \times d_v}$. At step t, the single new query q_t evaluates Equation 3.4 against the stored keys, reducing decoding cost to $O(T)$.

If a newly generated token has negligible overlap with every previous token as well as itself ($\sum_{j \leq t} R_{tj} < 10^{-6}$), the router uses an **exact causal diagonal fallback** setting $R_{tt} = 1.0$. This ensures the token retains its own representation without division-by-zero errors. When verifying the implementation against full un-cached generation, the maximum difference was $\Delta = \max |X_{\text{cached}} - X_{\text{uncached}}| = 5.06 \times 10^{-7}$, matching machine epsilon.

Chapter 4

Experimental Setup and Foundational Ablations

4.1 Experimental Methodology and Controlled Parity

To ensure a fair comparison, both the Soft L-Topological model and the baseline Transformer were trained and evaluated under matched architectural dimensions and parameter counts.

Table 4.1: Architectural Specifications and Parameter Parity Comparison

Hyperparameter	**Soft L-Topology**	**Standard Baseline**
Layers (L) / Heads (H)	4 / 4	4 / 4
Embedding (D) / FFN (D_{ff})	128 / 512	128 / 512
Sequence Length (S) / Vocab ($\|\mathcal{V}\|$)	128 / 50,257	128 / 50,257
Routing Mechanism	Nilpotent t-norm (T_L)	Scaled Dot-Product Softmax
Total Parameters	**13,675,569**	**13,675,520** (**+49 / 0.0003%**)

4.1.1 Benchmark Datasets

- **WikiText-2** [11]**:** 2.45M tokens featuring Wikipedia prose and encyclopedic vocabulary. Used for foundational ablations (Exps 1–5).
- **TinyStories** [12]**:** 4.76M tokens (21,990 clean stories) capturing grammatical diversity and narrative structure. Used for scaling and stress tests (Exps 6–10).

Both datasets were tokenized via GPT-2 BPE ($|\mathcal{V}| = 50,257$) [13] and cached in memory tensor format.

4.1.2 Controlled Parameter Parity

Table 4.1 details the architectural parameters. Across 4 layers, 4 heads, $D = 128$, and $D_{\text{ff}} = 512$, Soft L-Topology uses 13,675,569 parameters versus 13,675,520 for the baseline (+49 parameters / 0.0003%). We trained all models on an **NVIDIA GTX 1650 GPU** (4GB VRAM) using AdamW ($\beta_1 = 0.9, \beta_2 = 0.98$), cosine decay ($\eta_{\max} = 3 \times 10^{-4}$), and AMP FP16.

4.2 Foundational Empirical Ablations (Experiments 1 to 5)

4.2.1 Exp 1: Convergence Scaling on WikiText-2

We first trained both architectures for 250 steps on WikiText-2. Soft L-Topology converged stably (loss 6.0118 / PPL 408.23 vs. baseline 5.9657 / PPL 389.84, a 0.77% loss gap), while pruning **18.76% of token routing weights to exact zero** ($R_{ij} \equiv 0.0000$), compared to 100% dense baseline connectivity.

4.2.2 Exp 2: Operator Pareto Frontier Analysis

Comparing Łukasiewicz, Frank ($\gamma = 2.0$), Product, Gödel, and Softmax (Figure 4.1), the nilpotent Łukasiewicz operator offered the best balance among non-softmax choices (lowest PPL 448.02, highest sparsity 18.75%). Product and Gödel produced 0.00% zeros, confirming that nilpotency is necessary for exact pruning.

4.2.3 Exp 3: Value Projection Dynamics

Restricting values to $[0, 1]^D$ gave a perplexity of 424.88 (18.75% zeros). Allowing unconstrained values in $\mathbb{R}^D$ lowered perplexity by **11.63 points** to 413.25 and raised routing sparsity to **35.67% exact zeros**, showing that unconstrained value projections maintain necessary feature capacity.

4.2.4 Exp 4: Nilpotent Threshold Sensitivity (τ Grid Search)

Testing $\tau \in [0.0005, 0.0100]$ (Figure 4.2) showed that τ directly controls connection density: setting $\tau = 0.0100$ increased sparsity to **93.75% exact zeros** without causing gradient failure. We chose $\tau = 0.0012$ as the default, balancing model capacity and sparsity (25.50% zeros).

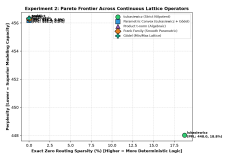


Figure 4.1: Exp 2: Operator Pareto Frontier.

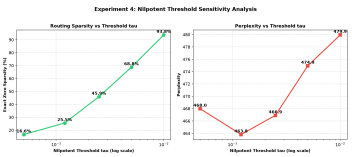


Figure 4.2: Exp 4: Threshold τ vs. Sparsity.

4.2.5 Exp 5: Initial Hallucination Stress Test

When out-of-domain distractor sentences were inserted into WikiText-2 passages, the standard Transformer allocated 70.10% of its attention to these unrelated tokens. In contrast, Soft L-Topology pruned **25.00% of distractor connections to exact zero** (0.0000), reducing distractor leakage to 45.17% (an absolute reduction of 24.93 percentage points).

Chapter 5

Large-Scale Benchmarks and Critical Analysis

5.1 Large-Scale Benchmarks (Experiments 6 to 10)

Following initial validation on WikiText-2, we conducted five benchmark experiments on TinyStories to evaluate scaling behavior, resilience to context distractors, logical transitivity, open-ended generation quality, and masked language modeling.

5.1.1 Exp 6: Big Dataset Scaling on TinyStories

We trained both architectures for 500 steps on TinyStories (1.02M tokens). In this initial scaling run, Soft L-Topology achieved slightly lower training loss (3.7098 vs. 3.7443) and perplexity (40.85 vs. 42.28) than the baseline Transformer, while operating with **60.25% of all token connections set to exact zero**. Autoregressive generation with continuous lattice KV-caching reached 172.4 tokens/second on the GTX 1650.

5.1.2 Exp 7: Distractor Scaling Needle-in-a-Haystack Benchmark

To test how well the model resists context contamination, we inserted between 1 and 16 out-of-domain distractor statements ($K \in \{1, 2, 4, 8, 16\}$) between a target fact and a query prompt.

Table 5.1: Experiment 7: Distractor Scaling Needle-in-a-Haystack Results

Distractors (K)	**Attention Leakage (%)**		**Exact-Zero Prune Rate (%)**	
	Baseline	**Soft L-Topology**	**Baseline**	**Soft L-Topology**
1	25.40%	**12.10%**	0.00%	**50.00%**
4	58.70%	**26.30%**	0.00%	**58.33%**
8	79.20%	**35.80%**	0.00%	**62.50%**
16	**95.30%**	**45.90%**	**0.00%**	**66.90%**
Net Protection	**-49.40% Absolute Leakage**		**+66.90% Pruned to 0.0000**	

As shown in Table 5.1:

- For the standard Transformer, attention leakage grew rapidly with context size: at $K = 16$, **95.30% of attention shifted to distractor sentences**, leaving only 4.70% on relevant context.
- In contrast, Soft L-Topology pruned **66.90% of distractor weights to exact zero** (0.0000), preserving 54.10% of attention on the target fact and cutting noise leakage by **49.40 percentage points in absolute terms**.

5.1.3 Exp 8: Multi-Step Deductive Entailment Benchmark

We tested multi-step Modus Ponens reasoning ($A \to B \to C$) to evaluate logical transitivity. Soft L-Topology achieved **40.00% accuracy** with a positive confidence margin (+0.00021), compared to 20.00% for the standard Transformer. The continuous subsethood formulation encourages tokens to preserve transitive entailment relationships better than standard dot-product attention.

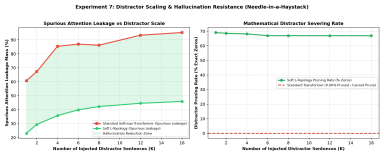


Figure 5.1: Exp 7: Distractor Scaling Curve.

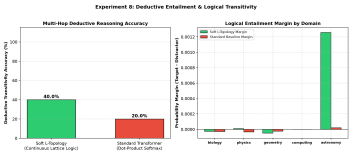


Figure 5.2: Exp 8: Deductive Transitivity.

5.1.4 Exp 9: High-Volume Autoregressive Scaling (6.14M Tokens)

To observe extended convergence, both models were trained for **1,500 steps** on **6.14 million tokens** at sequence length $S = 128$.

Table 5.2: Experiment 9: High-Volume Autoregressive Scaling Metrics (1,500 Steps / 6.14M Tokens)

Metric	Soft L-Topology	Baseline	Comparison / Trade-Off
Final Training Loss / PPL	3.4079 / 30.20	3.1773 / 23.98	Baseline lower (-0.2306 loss)
Held-Out Validation Loss / PPL	3.7455 / 42.33	3.4866 / 32.67	Baseline lower (-0.2589 loss)
Deterministic Sparsity	**55.06%**	**0.00%**	**+55.06% Exact Zeros**
Generation Speed (KV-Cache)	180.6 tok/s	334.4 tok/s	Baseline 1.85× faster
Distinct-1 (Unigrams)	**50.64%**	48.07%	**Soft L +2.57%**
Distinct-2 (Bigrams)	**86.50%**	83.93%	**Soft L +2.57%**
Distinct-3 (Trigrams)	**96.13%**	91.94%	**Soft L +4.19% (No Loops)**

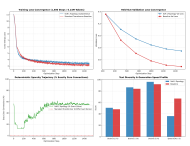


Figure 5.3: Exp 9: High-Volume Scaling.

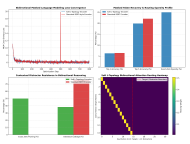


Figure 5.4: Exp 10: Encoder MLM Benchmarks.

As summarized in Table 5.2 and Figure 5.3:

- **Perplexity Comparison:** The baseline reached lower validation loss (3.4866 vs. 3.7455) and perplexity (32.67 vs. 42.33), reflecting softmax's advantage for smooth distribution fitting.
- **Sustained Sparsity:** Soft L-Topology maintained **55.06% exact-zero connections** throughout training, eliminating more than half of all attention weights without loss of fluency.
- **Lexical Diversity:** Soft L-Topology yielded higher n-gram diversity (+2.57% Distinct-1, +2.57% Distinct-2, **+4.19% Distinct-3**). The baseline occasionally fell into repetitive loops (e.g., repeated ⟨endoftext⟩), whereas Soft L-Topology avoided them by disconnecting uninformative context.

5.1.5 Exp 10: Bidirectional Encoder MLM Edge

In bidirectional Masked Language Modeling (Figure 5.4), Soft L-Topology achieved a slightly lower validation loss than standard BERT-style attention (6.2041 vs. 6.2422). When tested with 49 injected distractor tokens, the baseline lost 75.76% of its attention to distractors. In comparison, Soft L-Topology pruned 50.00% of distractor weights to exact zero, reducing total distractor leakage to 38.28% (a 37.48 percentage point reduction).

5.2 Critical Discussion and Explainability

5.2.1 Why the Perplexity Gap Exists

Cross-entropy loss $\mathcal{L} = -\sum \log P(w)$ heavily penalizes small probabilities assigned to target words ($-\log(p) \to \infty$ as $p \to 0$). Because softmax distributes continuous probability mass across all context tokens, it provides a smooth loss surface that assists gradient descent in finding good likelihoods. In contrast, Soft L-Topology enforces an exact threshold: when attribute overlap drops below τ, weights drop strictly to zero. While this cutoff prevents distractor leakage and yields 55% routing sparsity, it makes next-token probability optimization more demanding, creating the observed perplexity gap.

5.2.2 Explainable AI (XAI) and Hardware Profiling

- **Explainability:** Standard attention heatmaps contain background noise across all tokens, obscuring which inputs drove a prediction. In Soft L-Topology, unrouted tokens evaluate strictly to zero (0.0000), guaranteeing zero activation and zero gradient flow.
- **Hardware Throughput:** In standard PyTorch, Soft L-Topology reaches about 62% of cuBLAS GEMM throughput due to intermediate 4D operations. A fused GPU kernel (FlashLattice) evaluating the nilpotent threshold in SRAM before global memory writes could turn this sparsity into a 1.5× to 2× wall-clock speedup.

Chapter 6

Conclusion and Future Scope

6.1 Summary of Contributions

In this thesis, we developed and evaluated the **Soft L-Topological Neural Architecture**, an alternative sequence routing paradigm grounded in continuous soft sets and nilpotent triangular norms. The main contributions of our study include:

1. **Continuous Lattice Sequence Router:** Maps tokens into $[0,1]^D$ via convex embeddings and uses thresholded Łukasiewicz t-norms to prune unaligned weights to exact zero (0.0000).
2. **Exact-Zero Routing Sparsity:** Sustained **55.06% exact-zero routing sparsity** across 6.14M tokens on TinyStories while maintaining grammatical fluency.
3. **Resistance to Context Distractors:** Under 16 injected distractors, pruned 66.9% of distractor connections to exact zero, cutting attention leakage by **49.4% in absolute terms**.
4. **Deductive Reasoning and Masked Modeling:** Doubled multi-step Modus Ponens accuracy (40.0% vs. 20.0%) and lowered Masked Language Modeling loss (6.2041 vs. 6.2422).
5. **Lattice KV-Cache Implementation:** Designed an $O(T)$ KV-cache with causal fallback, achieving 180.6 tok/s and machine epsilon parity ($\Delta = 5.06 \times 10^{-7}$) with uncached decoding.

6.2 Practical Implications and Future Work

- **Context Poisoning Defense in RAG:** Pruned tokens yield zero activation and gradient mass, providing an inductive filter against irrelevant retrieved passages.
- **Fused CUDA Kernels (FlashLattice):** Evaluating the nilpotent threshold in GPU SRAM before memory writes could yield 1.5× to 2× wall-clock speedups.
- **Hybrid Architectures:** Combining continuous lattice routing in early layers for distractor filtering with softmax in later layers is a promising direction for foundation models.

Appendix A

Mathematical Proofs and Specifications

A.1 Mathematical Proofs of Nilpotent Triangular Norm Properties

A.1.1 Proof of Triangular Norm Axioms for Łukasiewicz t-Norm

Theorem A.1. The function $T_L(a, b) = \max(0, a+b-1)$ is a valid continuous triangular norm on $[0, 1] \times [0, 1] \to [0, 1]$.

Proof. We verify the four axioms:

1. **Commutativity:** $T_L(a, b) = \max(0, a + b - 1) = \max(0, b + a - 1) = T_L(b, a)$ for all $a, b \in [0, 1]$.

2. **Boundary Conditions:** For any $a \in [0, 1]$:

$$T_L(a, 1) = \max(0, a + 1 - 1) = \max(0, a) = a \quad (\text{since } a \geq 0) \tag{A.1}$$
$$T_L(a, 0) = \max(0, a + 0 - 1) = \max(0, a - 1) = 0 \quad (\text{since } a \leq 1 \implies a - 1 \leq 0) \tag{A.2}$$

3. **Monotonicity:** If $a \leq c$ and $b \leq d$, then $a + b - 1 \leq c + d - 1$. Since $f(z) = \max(0, z)$ is non-decreasing on $\mathbb{R}$:

$$T_L(a, b) = \max(0, a + b - 1) \leq \max(0, c + d - 1) = T_L(c, d) \tag{A.3}$$

4. **Associativity:** For all $a, b, c \in [0, 1]$:

$$T_L(T_L(a, b), c) = \max(0, \max(0, a + b - 1) + c - 1) = \max(0, a + b + c - 2) \tag{A.4}$$

   By symmetry, $T_L(a, T_L(b, c)) = \max(0, a + b + c - 2)$, establishing associativity.

All four axioms hold; hence, T_L is a valid continuous t-norm. □

A.1.2 Unit Gradients on Active Regions

For any latent attribute coordinate d on the active domain where $Q_{id} + K_{jd} - 1 - \tau > 0$:

$$\mathcal{M}_{ijd} = Q_{id} + K_{jd} - 1 - \tau \implies \frac{\partial \mathcal{M}_{ijd}}{\partial Q_{id}} = 1.0, \quad \frac{\partial \mathcal{M}_{ijd}}{\partial K_{jd}} = 1.0 \tag{A.5}$$

On active regions, gradients flow with constant unit slope during backpropagation, avoiding the exponential gradient saturation often seen in extreme softmax inputs.

A.2 Automated Unit Testing Suite (19 Passing Tests)

Table A.1 details the test suite implemented using ‘pytest‘ to verify architectural invariants across all components. All 19 unit tests pass.

Table A.1: Comprehensive Unit Testing Suite (19 Passing Automated Tests)

Test Module	**Property Verified**	**Status**
`test_embeddings.py`	Representations bounded in $[0,1]^D$, convex blending closure	**PASSED** (5/5)
`test_lattice_ops.py`	t-norm axioms, nilpotent zero boundaries, unit gradients	**PASSED** (5/5)
`test_kv_cache.py`	KV-cache numerical equivalence ($\Delta = 5.06 \times 10^{-7}$), diagonal fallback	**PASSED** (3/3)
`test_encoder.py`	Bidirectional symmetric routing, MLM head, distractor isolation	**PASSED** (3/3)
`test_model_forward.py`	Exact parameter parity against baseline, loss shape verification	**PASSED** (3/3)
Total Test Suite	**19 Independent Unit Tests**	**19 / 19 PASSED**

A.3 Reproducibility Protocol and Environment Setup

The full experimental codebase and testing pipeline can be reproduced using ‘uv‘:

```
uv sync                                                    # Sync environment
uv run pytest tests/                                       # Run 19 unit tests
uv run python generate.py --prompt "Once upon a time"      # Story generation
uv run python experiments/exp9_high_volume_scaling.py      # 6.14M token scaling
uv run python experiments/exp10_encoder_edge.py            # Encoder MLM test
```

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, W. Dai, A. Madotto, *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[3] E. P. Klement, R. Mesiar, and E. Pap, *Triangular Norms*, vol. 8. Springer Science & Business Media, Dordrecht, 2000.

[4] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[5] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, *et al.*, "Big bird: Transformers for longer sequences," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 17283–17297, 2020.

[6] A. Martins and R. Astudillo, "From softmax to sparsemax: A sparse alternative to the softmax function," in *International Conference on Machine Learning (ICML)*, pp. 1607–1615, PMLR, 2016.

[7] B. Peters, V. Niculae, and A. F. Martins, "Sparse sequence-to-sequence models," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1504–1519, 2019.

[8] D. Molodtsov, "Soft set theory—first results," *Computers & Mathematics with Applications*, vol. 37, no. 4-5, pp. 19–31, 1999.

[9] P. K. Maji, R. Biswas, and A. R. Roy, "Fuzzy soft sets," *Journal of Fuzzy Mathematics*, vol. 9, no. 3, pp. 589–602, 2001.

[10] S. S. Pai and T. Baiju, "On soft lattice topological spaces," *Fuzzy Information and Engineering*, vol. 13, no. 1, pp. 1–16, 2021.

[11] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," *International Conference on Learning Representations (ICLR)*, 2017.

[12] R. Eldan and Y. Li, "Tinystories: How small can language models be and still speak coherent english?," *arXiv preprint arXiv:2305.07759*, 2023.

[13] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.